

Passing Parity Is Not Preserving Behaviour: Oracle Escape in Function-Level LLM Code Migration

Simarjot Khanna
simarkhanna76@gmail.com
Independent Researcher
Canada

Abstract

Parity testing validates software modernization: the legacy system supplies every expected output, so the oracle is sound. One decision is delegated, increasingly to a language model and often the very model that translated the code: which inputs to try. How much divergence such suites let through is unmeasured, and whether a model validating its own translation misses more is unknown; we pre-registered that self-validation hypothesis. We measure both in Python-to-Java function migration with five models, each acting as both translator and validator, with expected values always from executing the legacy source and divergence established by differential execution on a large type-directed input set together with the generated suites. Of the parity suites that passed, 40.8% were on divergent translations, and 7.7% on translations that diverge on an input a reviewer would call unremarkable. We could not detect a self-validation penalty. Validator strength and translator subtlety dominate instead: misses are worst when a weak validator checks a strong translator's code, the configuration a cost-conscious pipeline prefers. A passing parity suite is evidence about where testing has looked, not a verdict of behavioural equivalence, and should be reported with its escape rate attached.

Keywords

software modernization, code migration, semantic preservation, test oracles, large language models, parity testing

ACM Reference Format:

Simarjot Khanna. 2026. Passing Parity Is Not Preserving Behaviour: Oracle Escape in Function-Level LLM Code Migration. In *Proceedings of the 2nd International Workshop on AI for Software Modernization (AISM '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3843775.3844550>

1 Introduction

Automated modernization has stopped being a translation problem and become a trust problem. Production pipelines translate COBOL to Java at enterprise scale [2] and upgrade Java repositories agentically [10]. Deployment turns on whether the output can be shown to behave like the system it replaces, not on whether it compiles.

Legacy estates rarely come with tests, so validation manufactures its own. The construction the field has converged on is *parity testing*: pick inputs, run the legacy system for the expected outputs, run

the migrated system, and compare. Symbolic execution has been used to pick those inputs for COBOL [7, 8], and an agentic loop has been proposed that searches for them under deterministic parity checks [5].

Nothing here depends on a model predicting what the program *should* do; the legacy system answers that. Exactly one thing is delegated, increasingly to a language model: the choice of *which inputs to try*. Throughout, the *translator* is the model that produced the translation and the *validator* is the model that chooses the parity inputs checking it; the two are often the same model.

We ask what that delegation costs: when an LLM-generated parity suite passes, how often is the migration divergent anyway, and does the answer depend on who wrote the suite? That measurement, against a ground truth the suites did not define, is the contribution.

Our design isolates the question. Every model acts as both translator and validator, so the matrix of pairings has a self-validation diagonal and a cross-validation off-diagonal, and expected values always come from executing the source, so the only free variable is where the validator looks. Pairing is partial: a divergent translation is scored by however many validators produced a usable suite for it, a median of 3 of 5, and the analysis is built around that. Section 3 gives the full design and the oracle's measured detection power.

What this study is and is not. We study Python-to-Java migration of short, self-contained functions. That is not a legacy estate, and we claim no numerical carry-over to a COBOL modernization. We chose this setting because it is the smallest one admitting a differential oracle at all: at repository scale there is no ground truth to score a parity suite against, which is why the question has not been answered. What transfers is the *principle* the oracle rests on: expected values are deferred to the legacy system while input selection is delegated. The shape of the result transfers too: a language pair has a gap class of its own, and generic testing is worst precisely there. The engineering does not transfer: a stateful estate needs mocked resources, replayed I/O and trace-level comparison [5, 8]. Its language pair will also have gap categories of its own, and a pipeline should go looking for them.

Contributions.

- An escape-rate measurement for migration validation: across 1583 validator-translator-problem cells, 40.8% of the LLM-generated parity suites that passed were on divergent translations (95% CI 32.8–49.2), and 7.7% on translations that diverge on an unremarkable input; under the independent oracle alone, 13.0%.
- Evidence on correlated blindness: no detectable self-validation penalty, from miss rates tested pairwise within translation.



This work is licensed under a Creative Commons Attribution 4.0 International License. *AISM '26*, Munich, Germany

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2987-4/2026/10
<https://doi.org/10.1145/3843775.3844550>

- A grounded taxonomy of what escapes, built from observed divergences rather than synthetic mutants.
- Three mitigations evaluated against the same ground truth, and a public replication package¹ that runs on one workstation.

2 Background and Related Work

LLM-assisted migration. Production modernization pipelines now combine static analysis with language models. IBM’s enterprise COBOL-to-Java pipeline pairs program analysis with LLMs and is evaluated by an automated LLM-as-judge on twenty GenAPP programs and ten customer apps [2]. MIGRATIONBENCH frames repository-level Java 8 upgrades as an agentic task over 5,102 repositories [10]. At the first edition of this workshop, migration and modernization tasks ranged from COBOL functionality extraction [14] to automated dependency upgrades [15].

Deciding that a migration preserved behaviour. Kumar et al. [8] generate COBOL unit tests by symbolic execution, mock external resource calls, and emit matching JUnit tests; the same group reports a testing framework built around this idea for IBM watsonx Code Assistant for Z [7]. A recent preprint describes the “Locksmith Loop” [5], which makes input selection *agentic*: an LLM loop searches for inputs that penetrate program branches, and accepted translations are those that match the source under deterministic parity checks. Across all of these the expected value is never in doubt; what differs is which inputs are tried. Our results project onto these systems directly: any input-selection mechanism, whether a symbolic executor, an agentic search loop or a prompted model, can be scored by the share of divergent translations its accepted suites let through, measured against divergences it did not itself establish. That is the evaluation this paper applies to LLM selectors, and nothing in its construction is specific to LLMs.

How much do such suites actually detect? Test-adequacy studies exist for code *generation* benchmarks. EvalPlus expands HumanEval’s tests by 80× and finds that pass@k drops by up to 19.3–28.9% once the extra inputs are applied [9], and Liu et al. [11] study the test adequacy of execution-based generation benchmarks directly. Translation corpora are weaker still: AVATAR pairs 9,515 Java and Python programs but supplies unit tests for only 250 [1]. Rabbi et al. [13] examine 6,164 translations and show that many reported *failures* are artifacts of misconfigured evaluation. That result is the mirror image of ours: they show reported failures overstate the defect count, we ask whether reported passes overstate correctness.

Blind spots in self-assessment. Language models assess their own output poorly. In a secure-code repair study, Gong et al. report that GPT-3.5 and GPT-4 “both perform poorly when repairing self-produced code, indicating self-repair ‘blind spots’” [6]. Zhao et al. [16] show that exposure to buggy code makes a model assert the erroneous behaviour and suppresses bug-finding tests. Both concern what the model *claims*, the expected value; migration validation removes exactly that freedom. Whether a blind spot survives when only *input selection* is left to the model is, to our knowledge,

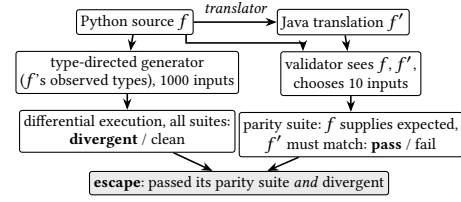


Figure 1: The pipeline, run per (validator, translator, problem) cell; an escape is a divergent translation whose parity suite passed.

untested, and it is the question this paper answers. Meta-validation of judges in low-data modernization settings has been raised at this workshop before [4]; we take the complementary execution-based route.

3 Study Design

The design is fixed in a pre-registration written before any model was called; it is included in the replication package together with its 10 recorded amendments, each with the reason and the direction of its effect.

Study design at a glance. 136 Python functions; 5 models; one translation per (model, problem) and one parity suite per (validator, translation), giving 1583 scorable validator-translator-problem cells (Figure 1). Two input sets play different roles. The 10 model-chosen inputs per suite, with expected outputs from executing the Python source, are the object under test. Differential execution on 1000 type-directed inputs per problem, unioned with the generated suites themselves, supplies the ground truth the suites are scored against. Both compare the translation to the source; they differ in role, not in kind. A translation *escapes* when its parity suite passes while the translation is divergent under the ground truth; a validator *misses* a divergent translation when its suite passes on it.

3.1 Set-up

We treat a Python function as the legacy source and Java as the modernization target. Java is the destination language of the enterprise pipelines this work is about; Python stands in for the legacy source, which in those pipelines is COBOL, as a dynamically typed language with a public corpus carrying executable tests. The corpus is the HumanEvalPack Python split [12], itself derived from HumanEval [3] (164 problems). Most HumanEval signatures carry no return annotation, so rather than parse types we *observe* them: the benchmark’s own test block is executed with the entry point replaced by a recording proxy, which yields the concrete argument and return types and the benchmark’s own inputs. 147 problems have types our marshalling layer supports; 11 more are dropped because their source is nondeterministic under repeated execution or because too few generated inputs are in domain, leaving 136.

Translations are produced by 5 models spanning distinct families: four run locally under Ollama (qwen2.5:7b-instruct, deepseek-coder:6.7b-instruct, mistral:7b-instruct-v0.2-q4_0, llama3:latest) and one is a hosted frontier model (gemini-3.6-flash-high, queried 7 August 2026). Every model acts as *both*

¹<https://github.com/s1mar/parity-escape>

translator and validator, giving a square matrix whose diagonal is self-validation. Decoding is fixed at temperature 0.2 and top-p 0.95, with seed 0 on the local route (the hosted route exposes no seed control). Every call allows 1,024 output tokens, one sample per cell, and up to 5 retries with exponential backoff that fire only on a transport failure or an empty response, never on content.

The translator is handed the source and an *exact* Java method signature derived mechanically from the observed types, and must implement it. Fixing the signature removes signature ambiguity as a confound and lets one reflective JSON harness serve every problem. The one exception: a translation failing to compile solely for missing imports gets a fixed import preamble, nothing else (27 of 680 attempted, 21 of them Llama 3's). Compile counts are post-repair, and excluding every repaired translation leaves the escape rate at 40.8%. Python `int` maps to Java `long`: no fixed-width type is faithful to an arbitrary-precision integer, and `long` is the conservative choice because it makes overflow *harder* to produce than `int` would.

3.2 The thing under test: a parity suite

Each validator receives the source and a candidate translation and returns 10 test *inputs* as JSON argument tuples. It is never asked for an expected value. The prompt shows both programs, states that expected values will come from executing the legacy source, and asks for the inputs “most likely to expose a behavioural difference between the two programs, if one exists”; the verbatim template is in the replication package. Expected values come from executing the Python source on those inputs. A translation *passes* if its output matches the source on every in-domain input. Inputs on which the source raises are out of domain; a suite leaving fewer than three in-domain inputs is recorded as degenerate and reported separately rather than counted as a silent pass.

3.3 Ground truth: the strong oracle

Divergence is decided by differential execution on 1000 type-directed inputs per problem, which include the benchmark's own inputs as a prefix, so the strong oracle is a strict superset of the benchmark-provided one. The generator's rule is fixed and mechanical, with one seed for the whole study. After the benchmark prefix, each generated input is flagged *boundary* with probability 0.30. A boundary input draws every argument from a fixed per-type catalogue: 15 small signed integers when the integer is a scalar argument, joined by 7 values near 2^{31} , 2^{63} and 10^{18} when it sits inside a list, where magnitude feeds the function's arithmetic rather than an allocation size. The float catalogue holds 14 values including signed zero, 10^{-9} and 10^{18} ; the string catalogue 17, including the empty string, whitespace, non-ASCII text and a 200-character string. Boundary lists have length 0, 1, 2 or 100, and with probability 0.4 are made all-equal, sorted, or reverse-sorted. A non-boundary input draws each argument from tiered magnitude ranges, small values weighted most; the catalogues and tiers are constants in the replication package. A translation is DIVERGENT if any in-domain input yields a different output. Integers compare exactly, floats to a relative tolerance of 10^{-6} , and a target exception where the source returned a value counts as divergence.

Resource exhaustion does not. Timeouts, out-of-memory conditions and oversize results are excluded on both sides, because a Java heap limit says nothing about whether behaviour was preserved. Excluding them can only remove candidate divergences, so every rate we report is a lower bound.

One class of artifact needs care, because a fixed signature can force a difference the model could not have avoided. Types are observed on the benchmark's own inputs, so a fuzz input can take the source somewhere those observations never went: `largest_divisor` returns an `int` on every benchmark input and `None` on a negative one, which no Java method returning a primitive can reproduce. We therefore treat any input on which the source returns a value that is not *representable in the declared Java type* as out of domain. Representability, not merely type: Python integers are unbounded and the declared type is `long`, so a source that legitimately computes a value beyond $2^{63} - 1$ would force the translation to differ whatever the model wrote, because we fixed the signature. That case accounts for 2,098 of 60,819 integer-returning source outputs across 25 problems, and all of them are excluded. The same rule covers the neighbouring cases: a source returning a float where `int` was inferred is out of domain rather than a divergence, no Python value is ever mapped to a Java `char`, and a problem whose element type cannot be resolved (an always-empty collection) never enters the corpus at all.

What the rule deliberately does *not* exclude is a source whose result is representable while the target's arithmetic diverges on the way there. Those are avoidable by a careful translation, so they are real defects. None of the width-related divergences we report has an out-of-range source value or an *ordinary* witness, so the conservative 7.7% rate of Section 4.2 does not depend on the integer mapping.

3.4 Controls

An instrument that detects nothing would report a clean zero escape rate and read as a null result, so the oracle's power is measured rather than asserted. **C1** round-trips every supported type through the marshalling harness and requires exact equality, isolating marshalling bugs from translation bugs. **C2** seeds single-token mutants of the legacy source (comparison-operator swaps, arithmetic swaps, constant increments) and reports what share the strong oracle kills, against what share the benchmark's own inputs kill on the same mutants. **C3** injects a known divergence into translations the oracle called clean and requires it to flip. **C4** executes every source twice and excludes any problem whose outputs differ, because parity against a nondeterministic source is meaningless. **C5** exploits a strict impossibility: the random baseline draws its inputs *from* the strong oracle's own input list, so a divergence it reports must also appear in the oracle's verdict for the same translation. Any violation would indict the fuzzer, the marshalling, the domain rules or the comparison predicate, exercised through two independently written code paths.

3.5 Metrics and statistics

We report two rates. **ESCAPE RATE** is the share of parity-passing translations that are divergent: the practitioner-facing number, though its denominator moves with translator quality. **MISS RATE** is

the share of *divergent* translations whose parity suite failed to catch them; it conditions on the translation, so a divergent translation is compared across the validators that produced a usable suite for it, at most one of which wrote it. That takes translator quality out of the contrast, which the escape rate cannot do.

Formally, index the scorable cells by (v, t, p) : validator v 's suite for translator t 's translation of problem p . Let $S(v, t, p) \in \{0, 1\}$ indicate that the suite passed, and let $D(t, p) \in \{0, 1\}$ indicate that the translation is divergent, meaning the strong oracle or any generated suite exhibited an in-domain input on which target and source outputs differ. Then

$$\text{escape rate} = \frac{|\{(v, t, p) : S=1, D=1\}|}{|\{(v, t, p) : S=1\}|} = \Pr[D=1 \mid S=1],$$

$$\text{miss rate} = \frac{|\{(v, t, p) : S=1, D=1\}|}{|\{(v, t, p) : D=1\}|} = \Pr[S=1 \mid D=1].$$

For the self-validation comparison of Section 4.3, the divergence indicator is *leave-one-out*: $D_{-v}(t, p) = 1$ only when the oracle or a suite written by some validator other than v witnessed a divergence, so a selector's own catches never help define the set it is scored against.

Confidence intervals come from a cluster bootstrap over problems (10,000 resamples; 2,000 for the regression of Section 4.3), since translations of one problem are not independent. Three hypothesis tests appear in this paper; the fixed-effects regression and the GEE reported beside RQ2 are estimation views of the same contrast, not further tests. The pre-registered one is a paired permutation test (10,000 permutations) reshuffling which validator carries the "self" label within a translation; we report it but do not rely on it, because validators are not exchangeable and it is therefore confounded. The test we rely on for RQ2 is an exact permutation over the 120 column permutations of the matrix, which holds every row and column marginal fixed. RQ4 uses an exact McNemar test on paired cells. Our pre-registration declared only the first of these; the second is a recorded amendment, written before any RQ2 number had been computed.

4 Results

4.1 Does the instrument work?

All five controls pass; two produce results in their own right.

Marshalling round-trips exactly for all 11 supported type tags, with 0 mismatches (C1). Nondeterminism is a declared exclusion reason that never fired: all 11 dropped problems fell to the in-domain floor and 0 to nondeterminism (C4). The strong oracle kills 94.0% of 151 seeded single-token mutants of the legacy source (C2), above the 90% floor fixed in advance. It flags 18 of 20 injected divergences (C3); both survivors are relational boundary flips that produced no divergence against 7,998 further differently-seeded inputs, so they are very likely equivalent mutants and C3 is better read as 18 of 18 non-equivalent injections. C5 exploits a strict impossibility, that the random baseline draws its inputs from the oracle's own list: over 510 scorable cells there are 0 violations.

Two of these numbers matter beyond instrument validation.

Source-language mutation adequacy does not predict translation detection. The benchmark's own test inputs, a mean

of 9 per problem, kill 87.4% of the same source mutants: the extra 1000 fuzz inputs buy 6.6 more points. Yet those same inputs detect only 50.3% of the 165 translations that actually diverge (83 of them). The comparison is deliberately between two different things, and that is the point: a suite scored against mutants of the *source* can look strong while missing nearly half the defects introduced by *translation*, because a single-token source mutant sits at a site the suite already exercises and a cross-language divergence sits where the two languages disagree. Two limits on that claim. We did not mutate the Java translations, so this is about the adequacy measure computed on the legacy program, not about mutation testing in general. And our operators are single-token; a richer operator set, or a system-level suite of the kind an enterprise estate carries, could close some of the gap. What the numbers rule out is treating a high source-side score as evidence that a migration has been validated.

Our own oracle is incomplete, and we report it rather than repair it. Counting translations that any method showed to be divergent, 315 in total, 151 were witnessed only by a generated parity suite and never by the 1000-input oracle, which independently identified 164. (That is one fewer than the 165 in Table 1: one divergent translation had no validator produce a scorable suite for it, so it cannot enter any cell-level count.) The reason is visible in the inputs: a model probes a bracket matcher with " ((a)) (b) ", a structured string the type-directed generator never emits. We did not strengthen the generator after seeing this, because tuning an instrument towards a larger headline number is not a defensible move. Instead, ground truth for RQ1 is the union D of the oracle and every suite, and RQ2 uses the leave-one-out rule D_{-v} of Section 3.5, the only rule clean in both directions; Section 4.3 reports the two one-sided alternatives beside it. 1000 random type-directed inputs miss divergences that ten chosen inputs find, so for this task input *selection* matters more than input *volume*.

4.2 RQ1: how much passes that should not

Across 1583 scorable validator-translator-problem cells, 425 of the 1041 passing cells sit on a divergent translation: an escape rate of 40.8% (95% CI 32.8–49.2). The unit is the cell, so a divergent translation contributes one escape per validator whose suite passed it. Under the oracle's witnesses alone the rate is 13.0% (135 cells); the union counts a divergence on *any* in-domain input as real. That is residual risk conditional on a pass: of the sign-offs this validation produced, this share was divergent. It is not the cost *caused* by delegating to a model: at the same budget the non-LLM baseline of Section 4.5 misses more, not fewer, of the same divergences.

That interval clusters on problems, treating the five models as fixed. Two checks bound the model dimension. Dropping each model in turn, as both translator and validator, the escape rate ranges from 34.8% to 52.3%, highest when Gemini is removed: no single model produces the effect. Resampling models as well as problems widens the interval to 20.7–61.2%. That dimension is coarse with five models; a reader who wants to generalise beyond this ensemble should use the wider interval.

The aggregate hides who wrote the suite. Escape rate by validator runs from 14.9% for Gemini to 50.8% for Mistral, with the three other open models between 42.2% and 50.3%. For a team generating

Table 1: Per model: translations the harness could execute, how many diverge from the source under the strong oracle, and the escape rate, under the union ground truth of Section 4.1, when that model acts as *validator*.

Model	usable	diverg.	diverg.%	escape%
Qwen2.5 7B	114	38	33.3	49.5
DS-Coder 6.7B	105	37	35.2	50.3
Mistral 7B	58	37	63.8	50.8
Llama 3 8B	97	39	40.2	42.2
Gemini 3.6 F	136	14	10.3	14.9

its parity inputs with a small local model, roughly one in two of the migrations it signs off is divergent.

Three things are excluded from the denominator rather than counted as passes, because a validator should not look safe by failing to produce a usable suite. 1026 of 2695 suites left fewer than three in-domain inputs and are recorded as degenerate; 85 (3.2%) could not be parsed into argument tuples at all; 1 more was lost to a failed model call. The extremes are far apart: Gemini produced 35 degenerate suites and 0 unparseable ones, Mistral 390 and 70. The exclusion is conservative: counting degenerate parseable suites as passes unless they caught something raises the escape rate to 48.5%.

More inputs help, but slowly. Taking prefixes of the same generated suites, the escape rate is 50.2% at $K=3$, 46.7% at $K=5$ and 40.8% at $K=10$. Tripling the budget removes about ten points of escape; one cannot test one’s way out at that rate.

One escape shows the shape of the problem. For median, Gemini validated its own translation with ten inputs that included [MAX], [MAX, MAX] and [MIN, MIN]: it was plainly thinking about 64-bit limits. It never tried [MAX, MIN], and on that input the source returns -0.5 while the translation returns 0.0 .

The mechanism is not the obvious one, and nothing overflows: $2^{63}-1$ has no exact double representation, so widening each long to double rounds it up to 2^{63} , the two extremes cancel to exactly zero, and the division returns zero. Python’s unbounded integers and true / do neither. The case looks like overflow and like truncating integer division, and is neither.

The suite passed. The divergence appears on 1 of 999 comparable inputs, so a slightly larger sample would not have found it by luck.

Finally, the conservative reading. Restricting to divergences that have at least one *ordinary* witness input, meaning no integer above 10^3 in magnitude anywhere in the input, no string longer than 25 characters or containing non-ASCII, and no list longer than 20 elements, the escape rate falls to 7.7% of passing cells (95% CI 4.2–11.9); 80 of the 425 escapes have such a witness, every one of them oracle-witnessed. So roughly one in thirteen parity-passing sign-offs is wrong on an input a reviewer would call unremarkable, and roughly two in five are wrong on some input.

4.3 RQ2: does it matter who wrote the suite

We could not detect it, and we had predicted we would. On divergent translations the raw miss rate is 54.3% under self-validation ($n=164$) against 48.4% under cross-validation ($n=690$), a gap of 5.9 points (95% CI -0.1 to 12.2, an interval that includes zero). That

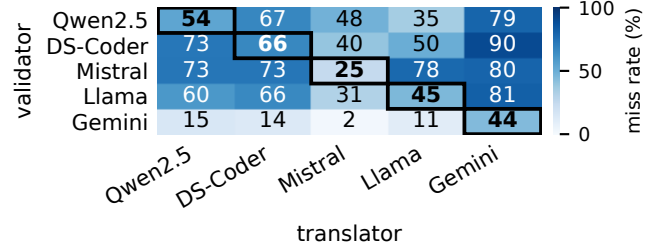


Figure 2: Miss rate (%) on *divergent* translations, each cell annotated with its value to the nearest whole point. Rows: the validator that chose the inputs; columns: the translator that wrote the code; boxed diagonal: self-validation. The visible structure is a pale bottom row (its own boxed self-cell aside) and a dark right column, not a dark diagonal.

raw gap is exactly the confounded comparison: the diagonal is populated by a different mix of validators and translators than the off-diagonal, so it cannot be read as an effect.

Two estimation views that do control it agree. A logistic regression over all 854 divergent cells, with fixed effects for validator identity and for translator identity and an indicator for self-validation, puts the odds ratio on missing under self-validation at 0.90 (95% CI 0.68–1.14, cluster bootstrap over problems, $p=0.38$); a logistic GEE with an exchangeable working correlation clustered on problem gives 0.89 ($p=0.29$). Both point slightly *below* one, meaning if anything a small self-validation advantage, and neither is distinguishable from no effect. These views handle the very unequal cell sizes natively, which the permutation below cannot.

The amended primary test tells the same story: the observed diagonal sits 4.7 points above the mean pseudo-diagonal, well inside the null range 40.1–58.9%, $p=0.18$ over all 120 permutations. Its caveat is the converse: column permutation does not weight the pseudo-diagonal quite as the true diagonal is weighted across cells of four to sixty-two observations, hence the estimation views beside it. The pre-registered paired test also agrees ($p=0.603$).

We report all of this as a failure to detect rather than a demonstration of absence: a penalty smaller than this design resolves could exist.

Ground truth here is the leave-one-out rule D_{-v} of Section 3.5. That matters, because the two obvious alternatives are biased in opposite directions, and we report both. Scoring on oracle-confirmed divergences only favours the random baseline, whose inputs are drawn from the oracle’s own list; scoring on the full union favours the model validators, whose own catches would help define the set. Leave-one-out has a residual cost of its own, in exactly the direction that matters here: it discards a catch that *only* the scored validator made, which is what a genuine self-recognition advantage would look like. That bias runs *against* the null we report, so it cannot have manufactured it, and we report the union beside it regardless. Under leave-one-out, 2 of 5 validators are worse on their own translations, mean difference -6.3 points; under the union, 2 of 5, mean -9.4 points. The exact matrix test is null under both ($p=0.18$ leave-one-out, $p=0.38$ union). The conclusion is the same under all three.

One more asymmetry could produce a spurious null: differential dropout between own and foreign code would leave the surviving diagonal and off-diagonal describing different populations. Pooled across all cells, dropout does differ: 34.7% of self cells are lost to degenerate or unparseable suites against 42.9% of cross cells. That gap is not a validator tendency: 3 of 5 validators drop out *more* on their own translations, the opposite direction.

Restricting to the 153 divergent translations that have both a usable self suite and at least one usable cross suite (the same translations the pre-registered paired test operates on), every translation contributes to both conditions and differential dropout cannot apply. On that matched subsample the miss rate is 52.3% self ($n=153$) against 48.6% cross ($n=364$), and the exact matrix test gives $p=0.42$. The null survives a design in which attrition is impossible.

One apparent exception: Gemini misses far more of the divergences in its own translations than in other models', which looks like self-blindness. It is not. Gemini's translations are hardest for *everyone* (other validators miss 84.2%, against 28.7–51.5% for the other four) and Gemini itself misses fewer of them than any other validator does. That is a column effect, which the permutation over columns is built to absorb.

What the matrix does show. Two large associations, neither of which is self-validation (Figure 2). We describe them rather than claim to have isolated them: this is an observational matrix, and translator identity bundles capability, defect mix and problem coverage together.

Along the rows, miss rate varies by *validator*: measured only on other models' translations, Gemini misses 11.0% while the four open models miss 56.9% to 75.6%. Along the columns, it varies by *whose translation is being checked*, from 28.7% for Mistral up to 84.2% for Gemini, and that ordering tracks translator quality: the divergences a better model leaves behind are the harder ones to catch.

Those two run in opposite directions, and the cell where both are unfavourable is the one a cost-conscious pipeline is most likely to build: a strong model to translate, a cheap model to check it; it is the worst cell we observe. DeepSeek-Coder validating Gemini's translations misses 89.6% of them ($n=48$). A 7B validator does not see roughly nine in ten of a frontier translator's divergences, and the reason is not that it is checking its own work.

4.4 RQ3: what diverges

The 5 arms attempted 680 translations, 5 per problem. 141 never compiled and 29 more compiled but produced no comparable output, leaving 510 executable. Compile failures are a visible defect that a pipeline catches for free, so they are excluded from every rate below rather than counted as escapes; the loss is uneven across arms and Table 1 gives it per model. Of the executable translations 165 (32.4%) diverge from the source under the strong oracle. The taxonomy classifies exactly these, because its rules read the triples of the oracle's differential runs; the larger union of Section 4.1 adds suite-only witnesses without such runs. Divergence tracks model capability in the expected direction: 10.3% for Gemini, 33.3% for Qwen2.5, 35.2% for DeepSeek-Coder, 40.2% for Llama 3, 63.8% for Mistral (Table 1).

Classifying each divergence by rule from its (input, source output, target output) triple gives Table 2. The assignment is mechanical. An ordered list of predicates over that triple is evaluated most specific first, and the first match assigns the category; a translation's category is the most frequent category over its divergent inputs. No model is involved, so the labels are deterministic, and the classifier ships with an audit mode that prints randomly sampled labelled divergences with their evidence so the labelling can be checked by hand.

The useful cut is not the individual categories but the split above them. 69.1% of divergences are *cross-language*: they arise where Python and Java disagree about what an operation means, so a direct or idiomatic Java implementation gets them wrong unless it explicitly emulates the Python semantics; a target-side exception where the source returned a value is counted here too. They are avoidable, which is why we count them as defects; they are simply not avoided by writing the obvious code. The remainder are ordinary logic errors, where the translation computes something else entirely.

A worked case shows what a characteristic cross-language divergence looks like, and how ordinary its witness can be, where the median escape of Section 4.2 was subtle. Asked for `greatest_common_divisor`, DeepSeek-Coder translated Euclid's algorithm line for line: the loop body becomes `a1 = temp % a1`, which is the obvious translation, and it is wrong. Python's remainder takes the sign of the divisor; Java's takes the sign of the dividend. On input `(-139, 6)` the source returns 1 and the translation returns `-1`, and the two differ on 221 of 1,000 comparable inputs, broadly those with a negative argument. Of the 4 usable parity suites for this translation, 2 caught it and 2 passed it, one of the passing two being DeepSeek-Coder's own: 10 plausible inputs are easy to choose without a negative first argument.

That split moves with model quality, and it moves the wrong way for anyone hoping better models will solve this: cross-language divergences are 56.8% of Mistral's failures and 78.6% of Gemini's. The ordering across the five is not strictly monotone, but the endpoints are far apart and the direction is clear. As translation improves, ordinary mistakes disappear and what remains concentrates in the semantic gap between the languages: fixed-width integers where Python has none, floored versus truncated division of negatives, case and index conventions on strings, and ordering of equal elements. Those are also the divergences the parity suites are worst at finding: on oracle-confirmed divergent cells the miss rate is 29.6% for cross-language categories against 17.4% for ordinary logic errors, and 71.9% for integer width alone; without integer width the remaining cross-language cells are missed 21.6%. The class that grows as translators improve is the class validation is least able to catch.

4.5 RQ4: what helps

We evaluated three mitigations against the same ground truth. Two do nothing.

Swapping to a merely *different* validator bought nothing measurable, because we could not detect a penalty attached to identity-matching for it to remove ($p=0.18$, Section 4.3). A failure

Table 2: Divergent translations by modal category, assigned by rule from the (input, source output, target output) triples of their divergent runs. 69.1% fall in cross-language classes.

category	n	%
other logic	51	30.9
string handling	44	26.7
integer width	17	10.3
negative division rounding	15	9.1
collection ordering	11	6.7
target exception:ArrayIndexOutOfBoundsException	9	5.5
target exception:StringIndexOutOfBoundsException	7	4.2
6 further categories	11	6.7
total	165	100.0

to reject is not proof of absence, and a penalty below the resolution of a five-model test could exist.

That must not be read as “the validator does not matter”, which our data contradicts loudly: measured on other models’ translations, validator miss rate runs from 11.0% to 75.6%. Choosing a *different* model is not the intervention. Choosing a *better* one is, and it is the largest effect in this study.

Telling the validator where to look does not help either. We reran self-validation with a prompt naming the cross-language classes explicitly: fixed-width overflow, floored versus truncated division of negatives, string indexing and case, and ordering of equal elements. Over 158 paired cells the miss rate moved from 52.5% to 53.8%, 1.3 points in the *wrong* direction; across these pairs the targeted prompt newly caught 1 and newly missed 3 (McNemar $p=0.625$). For this prompt, on these cells, naming the gap classes did not turn into inputs that land in one.

Adding cheap generated inputs does help. On 854 cells, a model-chosen suite misses 49.5% and 10 type-directed random inputs miss 59.5%, but the union of the two misses 40.9%, a reduction of 8.6 points. The two are complementary rather than redundant: 159 divergences were caught only by the model and 74 only by the generator.

One asymmetry qualifies every selector comparison here: except in the matched-budget union above, the model’s 10 inputs meet the generator’s 1000 at their deployed budgets, not at equal ones, and whether a much larger model-chosen budget would close the generator’s remaining margin is not answered by this design. At deployed budgets, what helped is a different *kind* of input, not a better-instructed model.

5 Discussion

Two mechanisms, and why the null is about only one. A self-validation penalty could arise from *capability*, since a weak model writes worse translations *and* worse suites, or from *correlation*, where a model’s particular misunderstanding of Java shapes both the defect it writes and the inputs it thinks worth trying. The within-validator and matrix tests separate them by holding validator capability fixed, and it is the correlation mechanism that they fail to find. No self-recognition is involved either way: each validator is prompted afresh, with no memory of the translation’s origin.

Source-side mutation score is the wrong yardstick here. The benchmark’s tests kill 87.4% of source mutants and detect 50.3% of real divergences; Section 4.1 explains why the two measure different things. A mutation score against the legacy program looks healthy right up to the point where it matters.

What we would tell a team doing this today. Two levers moved: validator strength (miss rates span 11.0–75.6% across validators), and 10 generated inputs beside the model’s own, which removed 8.6 points of miss rate for no model call yet do not substitute for a stronger validator’s far lower residual. Run both selectors: a model probes structured, meaning-carrying inputs a generator will not construct, and a generator probes magnitudes, empties and boundaries nobody writes by hand, so neither subsumes the other. A better prompt moved nothing we could measure. Be most careful exactly where intuition says to relax: a strong translator’s surviving defects are the subtle ones, and a cheap validator will not see them. That is about catching a divergence once it exists, not how often one exists: residual risk after a pass still runs from 32.2% on Gemini’s translations to 56.0% on Mistral’s, so the strongest translator remains the safest source, consistent with its lowest divergence rate in Table 1. Treat a passing suite as evidence about where you have looked, and do not let a source-side mutation score replace it.

6 Threats to Validity

The corpus is not a legacy estate. HumanEval functions are short, self-contained and pure; COBOL programs are none of these, so the *principle* transfers, not the engineering or the rates. Code size and complexity are likely to cut both ways. On larger units a model validator must reason about more paths and more state, while a type-directed generator needs only the signature; but the structured inputs and program states larger code requires are exactly what blind generation is worst at constructing. Which effect dominates cannot be answered in this design.

Are the divergences real defects, or unreasonable inputs? A translation may differ only on inputs the original author never contemplated. We take the modernization stance that behavioural equivalence with the legacy system is the requirement, undocumented inputs included, because legacy edge-case behaviour is routinely depended on downstream. The *ordinary*-witness rate in Section 4.2 is the conservative reading for readers who disagree.

Our oracle could be wrong in either direction. False divergences would inflate every rate: C1 round-trips every type exactly and resource exhaustion is excluded on both sides. False equivalences would deflate them, which C2 and C3 measure rather than assume. The oracle is finite, so a translation we call clean may diverge on an input we never tried, again making the rates lower bounds.

Non-termination is not counted. A two-second per-call limit cannot distinguish a target that loops forever from one that is merely slower than the source, so timeouts are excluded rather than counted as divergence. This is conservative in an uncomfortable direction: a migrated program that hangs where the original returned is arguably the most severe failure of all, and we exclude it. All 259 target executions that timed out where the source returned

fall in translations the oracle already calls divergent, so counting them as divergences changes no rate here.

Models and sampling. One sample per cell at temperature 0.2, four small open models and one hosted frontier model. Different or larger models may show different rates; one sample per cell cannot separate typical behaviour from a lucky draw. We report the frontier arm separately. Ollama served each tag at its library default quantisation, which may not match the reference checkpoints; the served weight digests are recorded in the replication package.

One shot, not an agentic search. Our validators are asked once for 10 inputs. Deployed systems search: symbolic execution drives COBOL test generation [8], and the Locksmith Loop [5] iterates until inputs penetrate branches. Coverage-driven search would fix an escape that is merely an unexplored path. It does not obviously fix one that sits on a path already taken, and 86.2% of the 130 oracle-witnessed escapes we traced (5 of the 135 resisted tracing) are of the second kind: the divergent input executes only source lines the passing suite had already executed. What those suites lack is not reach but the boundary value inside a line they already ran.

The prompt differs between conditions, unavoidably. A validator sees its own code on the diagonal and a stranger's off it, and those differ in length and style. The matched subsample removes differential *dropout* and the matrix test removes row and column composition, but neither removes the possibility that a weak model reasons less well about unfamiliar code.

Fixed signatures. Handing the translator the target signature makes our setting easier than practical migration, and removes a confound that would otherwise dominate: without it, an incompatible signature and a semantic divergence are indistinguishable at the harness boundary.

7 Conclusion

Migration validation asks a language model to decide where to look, then trusts the legacy system to say what it should have seen. The trusted half is sound; the risk sits in the delegated half, and not where we expected: we could detect no penalty for a model checking its own translation rather than a stranger's. The risk is input selection treated as a detail when it is the whole of the evidence, and getting cheaper exactly as the translations it must check get subtler. What helped was a stronger validator, and a handful of inputs from a generator that does not share the model's sense of a plausible input. A passing parity suite is real but weak evidence, and belongs in a report with its escape rate attached, not as a verdict.

Data Availability

The replication package is publicly available at <https://github.com/s1mar/parity-escape>. It contains the frozen pre-registration with its amendments, the verbatim prompts, all translations and parity suites, the oracle and control outputs, and the scripts reproducing every number here. The corpus is public [3, 12] and the experiment runs on one workstation with locally hosted models.

Use of Generative AI

Language models are the object of study here: all translations and parity suites were machine-generated, as described in Section 3. AI assistance was also used in implementing the experimental harness and in preparing the manuscript. The design, the analysis and the claims are the author's.

References

- [1] Wasi Uddin Ahmad, Md Golam Rahman Tushar, Saikat Chakraborty, and Kai-Wei Chang. 2023. AVATAR: A Parallel Corpus for Java-Python Program Translation. In *Findings of the Association for Computational Linguistics: ACL 2023*. 2268–2281. doi:10.18653/v1/2023.findings-acl.143
- [2] Venkatesan Chakaravathy, Anamitra Roy Choudhury, Dinesh Garg, Vini Kanvar, Shivmaran Pandian, Aditya Raghuvanshi, Yogish Sabharwal, and Amith Singhee. 2026. Enterprise-Scale COBOL-to-Java Translation: LLMs Augmented with Program Analysis. In *Proceedings of the 48th International Conference on Software Engineering (ICSE), Software Engineering in Practice*. doi:10.1145/3786583.3786915
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374
- [4] Ora Fandina, Gal Amram, Eitan Farchi, Shmulik Froimovich, Raviv Gal, Wesam Ibraheem, Rami Katan, Alice Podolsky, and Orna Raz. 2025. Vintage Code, Modern Judges: Meta-Validation in Low Data Regimes. In *Proceedings of the 1st International Workshop on AI for Software Modernization (AIISM)*. doi:10.1109/ASEW67777.2025.00015
- [5] Andras Ferenczi, Jordan Docherty, Mariya Bessonov, Matthew Findlay, and Krishna Lingamneni. 2026. Agentic Method for Deterministic Validation of Legacy Code Migration. *arXiv preprint arXiv:2607.28271* (2026). doi:10.48550/arXiv.2607.28271
- [6] Jianian Gong, Nachuan Duan, Ziheng Tao, Zhaohui Gong, Yuan Yuan, and Minlie Huang. 2025. How Well Do Large Language Models Serve as End-to-End Secure Code Agents for Python?. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE)*. doi:10.1145/3756681.3756984
- [7] Sandeep Hans, Atul Kumar, Toshikai Yasue, Kouichi Ono, Saravanan Krishnan, Devika Sondhi, Fumiko Satoh, Gerald Mitchell, Sachin Kumar, and Diptikalyan Saha. 2025. Automated Testing of COBOL to Java Transformation. *arXiv preprint arXiv:2504.10548* (2025). doi:10.48550/arXiv.2504.10548
- [8] Atul Kumar, Diptikalyan Saha, Toshikai Yasue, Kouichi Ono, Saravanan Krishnan, Sandeep Hans, Fumiko Satoh, Gerald Mitchell, and Sachin Kumar. 2024. Automated Validation of COBOL to Java Transformation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), Tool Demonstrations*. doi:10.1145/3691620.3695365
- [9] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. *Advances in Neural Information Processing Systems (NeurIPS)* (2023).
- [10] Linbo Liu, Xinle Liu, Qiang Zhou, Lin Chen, Yihan Liu, Hoan Nguyen, Behrooz Omidvar-Tehrani, Xi Shen, Jun Huan, Omer Tripp, and Anoop Deoras. 2025. MigrationBench: Repository-Level Code Migration Benchmark from Java 8. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD)*. doi:10.1145/3770855.3817526
- [11] Xiangyue Liu, Xiaobing Sun, Lili Bo, Yufei Hu, Xinwei Liu, and Zhenlei Ye. 2025. Evaluating the Test Adequacy of Benchmarks for LLMs on Code Generation. *Journal of Software: Evolution and Process* 37 (2025). doi:10.1002/smr.70034
- [12] Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. 2024. OctoPack: Instruction Tuning Code Large Language Models. In *The Twelfth International Conference on Learning Representations (ICLR)*.
- [13] Fazle Rabbi, Soumit Kanti Saha, and Jinqui Yang. 2026. Beyond Translation Accuracy: Addressing False Failures in LLM-Based Code Translation. *arXiv preprint arXiv:2605.02195* (2026). doi:10.1145/3805760.3814890
- [14] Asha Rajbhoj, Akanksha Somase, Tanay Sant, Ajim Pathan, Purvesh Doud, and Vinay Kulkarni. 2025. Leveraging LLM for Software Modernization: COBOL Functionality Extraction Case Study. In *Proceedings of the 1st International Workshop on AI for Software Modernization (AIISM)*. doi:10.1109/ASEW67777.2025.00012
- [15] Vali Tawosi, Salwa Alamir, Xiaomo Liu, and Manuela Veloso. 2025. LLM Agents for Automated Dependency Upgrades. In *Proceedings of the 1st International Workshop on AI for Software Modernization (AIISM)*. doi:10.1109/ASEW67777.2025.00016
- [16] Junda Zhao, Shurui Zhou, and Eldan Cohen. 2026. Evaluating and Mitigating the Misguidance Effect of Buggy Code in LLM-Generated Unit Tests. *Proceedings of the ACM on Software Engineering (PACMSE)* 3, ISSTA, Article ISSTA113 (2026).